GRC WISDOM

Risk Management Module

Full Implementation & Enhancement Specification

VS Code Agent Handoff Document

Document Version	v1.0 — MVP Implementation
Prepared For	Development Team / VS Code Agent
Module	Risk Management — GRC Wisdom Suite
Frameworks	For all Frameworks
Stack	React 19 + Vite + TypeScript + Tailwind v4
Date	May 2026

1. Document Purpose & Scope

This document is a complete, implementation-ready specification for making the GRC Wisdom Risk Management Module fully functional. It is written in plain language so that auditors, consultants, and developers share the same understanding of what is being built and why each feature matters.

The document covers every gap identified in the current codebase, specifies exactly what must be built, how it should behave, what the user should see, and what the code must do — from the React front-end through to the data layer.

1.1 What the Module Must Achieve

A fully functional Risk Management Module for enterprise GRC must allow the following without any manual workarounds:

- A Risk Analyst can log in, create a new risk with full identification, analysis, and scoring data, and submit it for approval — all within one workflow.
- A Compliance Officer can view every risk in the register filtered by department, category, or severity, and export a board-ready report in one click.
- An Internal Auditor can trace every change ever made to any risk — who changed what, when, and why — without leaving the platform.
- A Risk Owner receives a notification when a KRI attached to their risk breaches its amber or red threshold, and can log a response directly from the alert.
- The system automatically calculates the Inherent Risk Score, applies the Control Rating from linked controls, and derives the Residual Risk Score — with no manual calculation required.
- Files (policies, evidence, third-party reports) can be attached manually or extracted automatically from uploaded documents using AI parsing.

1.2 Who This Document Is For

Audience	Role in This Project	How to Use This Document
VS Code Agent / Developer	Implements all features described	Follow Section 3 onwards sequentially. Each feature has exact implementation instructions.
GRC Consultant / Risk Analyst	Validates feature completeness	Review Section 2 (current state), Section 4 (workflow logic), and Section 6 (scoring engine).
Internal Auditor	Confirms audit trail requirements	Focus on Section 8 (audit trail) and Section 9 (file management).
Product Owner	Approves scope and prioritises	Review Section 2 gap analysis and Section 10 (phased roadmap).

2. Current State — Honest Gap Analysis

The current application is a well-designed front-end prototype. The visual language is professional and consistent with enterprise GRC standards. However, it is not yet functional as a GRC tool. Every page either shows mock data, has no backend connection, or is missing critical workflow logic.

The table below maps every module in the current codebase to its honest status, the specific gap, and the priority for fixing it.

Module	Status	What Is Missing	Why It Matters (GRC Impact)	Priority
Risk Register	Mock data only	No API calls. Refresh clears all risks. No filter/search logic wired.	Auditors cannot rely on a register that disappears. Core of the entire module.	**P0 — Critical**
Risk Scoring	Incomplete	getRiskScore = likelihood x impact only. No CIA weighting, no Control Rating, no Residual Risk formula.	Every risk score in the app is incorrect by GRC standards. Inherent vs Residual distinction is lost.	**P0 — Critical**
Authentication	Not built	No login, no JWT, no RBAC. Alice Smith is hardcoded. Any user sees everything.	A GRC platform with no auth is unusable in any enterprise. Blocks integration with GRC Wisdom.	**P0 — Critical**
Control Library	Read-only	No create/edit control workflow. Control effectiveness not linked to risk scoring.	Controls are the mechanism for reducing residual risk. Without this link, the scoring engine is broken.	**P0 — Critical**
Treatment Monitor	UI only	Progress bars are static. No task assignment, no approval gates, no treatment state machine.	ISO 31000 requires treatment to be tracked, assigned, approved, and evidenced. None of this works.	**P1 — High**
KRI Module	Static display	No threshold engine, no alert dispatch, no trend charting, no escalation chain.	KRIs are the early warning system. Without live thresholds, they are decorative only.	**P1 — High**
Documents	Placeholder	No file upload, no storage, no AI extraction, no categorisation. Three static cards only.	Auditors need evidence attached to risks. Without this, no finding can be substantiated.	**P1 — High**
Reports	Tiles only	Twelve report types are shown but none generate output. No PDF, no Excel, no filter.	Board and management cannot receive risk reporting without a working report engine.	**P2 — Medium**
Approvals	Not built	No approval workflow, no sign-off chain, no notification on submission.	Risk submissions must be approved before they are active. This is required by every GRC framework.	**P1 — High**

Audit Trail	Not built	No logging of create, edit, approve, or delete actions. No change history viewer.	SOX, ISO 27001, and GDPR all require immutable records of who changed what and when.	**P0 — Critical**

3. Target Architecture — What to Build

The following architecture must be implemented to make the module fully functional. The front-end already exists. What is needed is a proper state management layer, a backend API, a database, and a file handling service.

3.1 Front-End Architecture (React + TypeScript)

The current DataContext.tsx file uses in-memory mock data. This must be replaced with a proper API client layer. The architecture below shows how every page connects to real data:

State Management Pattern

Replace the single DataContext with a modular API service layer. Each domain (risks, controls, KRIs, treatments, documents) gets its own service file that calls the backend.

- src/services/riskService.ts — all risk CRUD operations and scoring
- src/services/controlService.ts — control library operations
- src/services/treatmentService.ts — treatment plan workflow
- src/services/kriService.ts — KRI threshold monitoring
- src/services/documentService.ts — file upload and AI extraction
- src/services/authService.ts — login, token management, RBAC
- src/store/DataContext.tsx — refactored to call services, not mock data

API Client Configuration

> *Implementation instruction: Create src/lib/apiClient.ts. Use Axios with a base URL from environment variable VITE_API_BASE_URL. Add a request interceptor that reads the JWT from localStorage and adds Authorization: Bearer <token> to every request. Add a response interceptor that handles 401 errors by redirecting to /login.*

3.2 Backend API (Node.js / FastAPI)

A REST API backend is required. The agent may choose Node.js with Express or Python with FastAPI. The endpoints below must be implemented regardless of the framework chosen.

Method	Endpoint	What It Does	Request Body	Response
POST	/api/v1/auth/login	Authenticate user, return JWT	email, password	token, user object, roles[]
GET	/api/v1/risks	List all risks (filtered)	Query: department, level, status	risks[], pagination
POST	/api/v1/risks	Create new risk	Full risk object	Created risk with computed scores

PUT	/api/v1/risks/:id	Update risk fields	Partial risk object	Updated risk, audit log entry
POST	/api/v1/risks/:id/submit	Submit risk for approval	submitter_note (optional)	Updated status, notifies approver
GET	/api/v1/controls	List all controls	Query: type, nature	controls[], effectiveness data
POST	/api/v1/controls	Create new control	Full control object	Created control
GET	/api/v1/kris	List KRIs with current values	Query: risk_id, status	kris[], RAG status, trend
POST	/api/v1/documents/upload	Upload file, trigger AI extraction	multipart/form-data	document_id, extracted fields
GET	/api/v1/audit-logs	Retrieve audit trail	Query: entity_id, entity_type	audit_entries[], actor details
GET	/api/v1/reports/:type	Generate and return report	Query: filters, format (pdf/xlsx)	Binary file stream

3.3 Database Schema

The following database tables are required. Use PostgreSQL. Every table includes a tenant_id column to support multi-tenancy for GRC Wisdom integration.

Core Tables

Table	Key Columns	Type	Purpose & Notes
risks	id, tenant_id, code	UUID primary keys	Core risk entity. Includes all scoring fields: cia_c, cia_i, cia_a, inherent_score, control_rating, residual_score, risk_level, treatment_strategy, status.
controls	id, tenant_id, code	UUID primary keys	Control library. Stores type (Preventive/Detective/Corrective), nature (Manual/Automated), design_effectiveness (0-100), operating_effectiveness (0-100).
risk_control_mappings	risk_id, control_id, weight	Junction table	Links risks to controls. The weight column (0.0 to 1.0) determines how much each control contributes to reducing the residual score.
treatment_plans	risk_id, strategy, status	Workflow entity	One per risk. Status flows: Draft > Submitted > Approved > In Progress > Closed. Stores assignee_id, approver_id, deadline, progress_pct, evidence_urls array.

kris	risk_id, thresholds	Monitoring entity	Stores red_threshold, amber_threshold, green_threshold, actual_value, and last_updated. Triggers alert when actual_value exceeds amber or red threshold.
documents	risk_id, file_path, type	File reference	Metadata for uploaded files. Stores original filename, storage path (S3 key or local), file type, upload timestamp, uploaded_by, and AI extraction status.
audit_logs	entity_id, action, actor_id	Immutable append-only	Records every create, update, submit, approve, reject, and delete action. Stores old_value and new_value as JSONB. No UPDATE or DELETE is ever permitted on this table.
users	id, email, role	Auth entity	Stores hashed password, role (risk_analyst, compliance_officer, auditor, risk_owner, cro, admin), department_id, and org_unit_id for row-level security.
regulatory_mappings	risk_id, framework	Tag table	Maps each risk to regulatory frameworks: ISO 27001, GDPR, PDPL, SOX, NIST, COSO, COBIT. Enables compliance reporting by framework.

4. Risk Scoring Engine — Full Specification

This is the most important technical section of the document. The current scoring logic in src/lib/risk-utils.ts is incomplete and must be replaced entirely. The replacement must implement the full ISO 31000 / ISO 27001 aligned scoring pipeline described below.

> *Plain language explanation: Think of risk scoring like a medical diagnosis. First you assess how sick a patient is without any treatment (Inherent Risk). Then you factor in the medication they are already on (Controls). What remains is how sick they still are despite treatment (Residual Risk). The current app only does the first step, and even that is too simple.*

4.1 The Three-Stage Scoring Pipeline

Every risk in the system must go through three calculations automatically. These run server-side when a risk is created or updated, and the results are stored in the database.

Stage 1 — Inherent Risk Score (IRS)

The Inherent Risk Score represents the raw exposure before any controls are applied. It uses the CIA triad (Confidentiality, Integrity, Availability) to weight the impact score.

Component	Specification
Likelihood	Integer from 1 (Rare) to 5 (Almost Certain). Set by the risk analyst on the risk form using a slider.
Impact	Integer from 1 (Insignificant) to 5 (Catastrophic). Set by the risk analyst on the risk form using a slider.
CIA Score	Three separate sliders for Confidentiality (C), Integrity (I), Availability (A), each rated 1-5. The combined CIA value = (C + I + A) / 3, rounded to 2 decimal places.
IRS Formula	***IRS = Likelihood x Impact x (CIA Score / 5)*** This yields a score from 1 to 125. Apply the following banding:
Score Bands	1 – 20 → LOW (Green): Risk is within acceptable tolerance. Monitor only. 21 – 50 → MODERATE (Amber): Requires treatment planning and quarterly review. 51 – 80 → HIGH (Orange/Red): Requires immediate treatment plan with named owner. 81 – 125 → CRITICAL (Dark Red): Escalate to CRO/Board. Treatment must begin within 30 days.

Stage 2 — Control Rating (CR) Calculation

When controls are linked to a risk in the Risk Detail page, the system must calculate a composite Control Rating. This represents how effective the existing controls are at reducing the risk.

Component	Specification

Design Effectiveness	How well the control is designed to address the risk. Scored 0-100. Stored per control in the controls table.
Operating Effectiveness	How consistently the control is operating in practice. Scored 0-100. This is what internal auditors test.
Control Score per Control	Single Control Score = (Design Effectiveness x 0.4) + (Operating Effectiveness x 0.6). Operating effectiveness is weighted higher because a well-designed but poorly-operated control provides little real protection.
Composite CR (Multiple Controls)	Composite CR = Weighted average of all linked control scores, using the weight column in risk_control_mappings. Result expressed as a decimal: 0.00 (no control) to 1.00 (perfect control).
CR Reduction Factor	The residual multiplier = 1 - CR. Example: CR of 0.70 means controls reduce the risk by 70%. Residual multiplier = 0.30.

Stage 3 — Residual Risk Score (RRS)

The Residual Risk Score is the risk that remains after controls are applied. This is the most important number for the risk register, treatment decisions, and board reporting.

RRS = IRS x (1 - CR) | Post-Treatment RRS = RRS x (1 - Treatment Progress %)

Example: A risk with IRS = 75 (HIGH), linked controls giving CR = 0.60, and a treatment plan 50% complete:

- Inherent Risk Score (IRS) = 75 — HIGH
- Control Rating (CR) = 0.60 (controls are 60% effective)
- Residual Risk Score (RRS) = 75 x (1 - 0.60) = 75 x 0.40 = 30 — MODERATE
- Post-Treatment RRS = 30 x (1 - 0.50) = 30 x 0.50 = 15 — LOW

4.2 Exact Code Change Required in risk-utils.ts

Implementation instruction: The entire content of src/lib/risk-utils.ts must be replaced with the following function signatures and logic. The file must export all three scoring functions so they can be called from the DataContext, the NewRisk form, and the RiskDetail page.

```
// src/lib/risk-utils.ts  —  Full replacement

export function computeInherentRiskScore(
  likelihood: number, impact: number,
  cia_c: number, cia_i: number, cia_a: number
): number {
  const ciaScore = (cia_c + cia_i + cia_a) / 3;
  return parseFloat((likelihood * impact * (ciaScore / 5)).toFixed(2));
}

export function computeControlRating(controls: ControlMapping[]): number {
  if (!controls.length) return 0;
  const totalWeight = controls.reduce((s, c) => s + c.weight, 0);
  const weightedScore = controls.reduce((s, c) => {
    const cs = (c.design_eff * 0.4 + c.operating_eff * 0.6) / 100;
```

```
    return s + cs * c.weight;
  }, 0);
  return parseFloat((weightedScore / totalWeight).toFixed(4));
}
```

5. Page-by-Page Implementation Instructions

This section provides exact, actionable instructions for every page in the application. Follow these in order. Each subsection tells the developer what currently exists, what must change, and what the user must experience after the change.

5.1 Dashboard (src/pages/Dashboard.tsx)

What must change

- Replace all mock data computations with real API calls to GET /api/v1/dashboard/summary
- Add a Risk Heat Map widget: a 5x5 grid where the X axis is Likelihood (1-5) and Y axis is Impact (1-5). Each cell is coloured green (low), amber (moderate), red (high/critical). Each risk appears as a dot on the grid, clickable to navigate to that risk's detail page.
- The 'Top Enterprise Risks' list must be sorted by Residual Risk Score descending, not by creation date.
- The 'Inherent Department Stats' bar chart must pull real department aggregates from the API, not mock counts.
- Add a Post-Treatment Risk donut chart that shows the risk distribution after treatment progress is factored in.

User experience after change

When a CRO opens the dashboard, they see live data. The heat map immediately shows where risks cluster. Clicking a dot on the heat map navigates to that risk. The three donut charts show the journey from inherent to residual to post-treatment risk.

5.2 Risk Register (src/pages/RiskRegister.tsx)

What must change

- Replace mock risks array with API call: GET /api/v1/risks with pagination (page, limit query params).
- Wire the Filter button to open a slide-over panel with filter fields: Department (dropdown), Division (dropdown), Risk Level (multi-select: Low/Moderate/High/Critical), Stage (multi-select), Treatment Strategy (multi-select), Date Range (created_at).
- Wire the Create Risk button to navigate to /risks/new.
- Add column sorting: clicking any column header sorts the table by that column ascending/descending. Show a sort indicator arrow.
- Add inline search: a text input that filters the visible table rows by risk title or code without a page reload.
- The Treatment Progress bar must show the real progress_pct from the treatment_plans table, not a hardcoded value.
- The Actions column (currently shows '—') must have a dropdown with: View Detail, Edit, Submit for Approval, Link Control, Create Treatment Plan.

Auditor language note

In GRC language, the risk register is the primary source of truth. Every action taken on a risk — creation, editing, approval, control linkage — must be traceable back to a named individual with a timestamp. The register is what an auditor examines first during any assurance engagement.

5.3 New Risk Form (src/pages/NewRisk.tsx)

What must change

- The form must be split into clearly labelled steps that map to GRC process: Step 1 — Risk Identification, Step 2 — Risk Analysis, Step 3 — Risk Assessment, Step 4 — Review and Submit.
- Step 1 must include: Risk Title (text), Risk Description (textarea), Division (dropdown from org units), Department (dropdown filtered by division), Risk Owner (user search/select), Process (dropdown), Objective (text), Risk Category (Strategic/Operational/Financial/Compliance/Reputational — dropdown), Root Cause (multi-select), Risk Consequences (multi-select).
- Step 2 must include: CIA sliders — Confidentiality, Integrity, Availability (each 1-5 with labels: 1=Negligible, 2=Minor, 3=Moderate, 4=Major, 5=Severe). Treatment Strategy selection (Avoid, Mitigate, Transfer, Accept) with a plain-language tooltip explaining each.
- Step 3 must show the Likelihood slider (1-5, labelled: Rare/Unlikely/Possible/Likely/Almost Certain) and Impact slider (1-5, labelled: Insignificant/Minor/Moderate/Major/Catastrophic). As the user moves either slider, the Inherent Risk Rating gauge must update in real time using the computeInherentRiskScore function.
- Step 4 must show a full summary of all entered fields before the user submits. Two buttons: Save as Draft (POST /api/v1/risks with status=draft) and Submit for Approval (POST /api/v1/risks then POST /api/v1/risks/:id/submit).
- Regulatory framework tags: a multi-select chip input on Step 1 allowing the user to tag the risk with applicable frameworks (ISO 27001, GDPR, PDPL, SOX, NIST, COBIT).

Treatment strategy plain-language tooltips — exact copy

Strategy	Tooltip Text (show on hover/click of info icon)
Avoid	We stop doing the thing that creates this risk entirely. Example: We stop processing credit card payments in-house to avoid PCI-DSS exposure. Use this when the risk is critical and no control can bring it to an acceptable level.
Mitigate	We put controls in place to reduce how likely this risk is to happen, or how bad it would be if it does. This is the most common strategy. Example: We implement multi-factor authentication to reduce the likelihood of a data breach.
Transfer	We shift the financial impact of this risk to a third party — usually through insurance or a contract. Important: this does not eliminate the risk. We are still responsible. Example: We buy cyber insurance to cover the financial cost of a breach.
Accept	We acknowledge this risk exists and decide to live with it — usually because it is low-level, or the cost of treating it is greater than the potential loss. This decision must always be formally documented and signed off by a named manager.

5.4 Risk Detail Page (src/pages/RiskDetail.tsx)

What must change

- The Risk tab must show all identification and analysis fields from the risk record — not just a subset. Include Division, Department, Process, Objective, Risk Owner (with avatar), Description, Root Cause, Consequences, Category, and Regulatory Tags as coloured chips.
- Display both Inherent Risk Rating (gauge chart) and Residual Risk Rating (gauge chart) side by side. Show the score number below each gauge. Below both gauges, show a simple sentence: 'Controls have reduced this risk from HIGH (IRS: 75) to MODERATE (RRS: 30).'
- The Control tab must list all linked controls with their design effectiveness, operating effectiveness, weight, and individual control score. Add a Link Control button that opens a modal to search and link controls from the control library. Show a composite Control Rating summary at the top of the tab.
- The Treatment tab must show the active treatment plan: strategy badge, current status badge, assigned owner, approver, deadline, progress bar (editable if user is the assignee), evidence files list with upload button, and action buttons (Submit Update, Request Approval, Mark Complete).
- Add a fourth tab: Activity Log — shows the audit trail for this specific risk in reverse chronological order. Each entry shows: timestamp, actor name and role, action taken (Created / Updated Field X: Old Value to New Value / Submitted / Approved / Rejected), and any attached comment.
- The Edit button must open the risk form pre-populated with existing values. Only users with role risk_analyst, compliance_officer, or admin may edit. Risk Owner may only edit their own risks.
- The Create Task button (currently in the header) must create a treatment action item linked to the treatment plan and assign it to a named user.

5.5 Control Library (src/pages/ControlLibrary.tsx)

What must change

- Replace static controls array with API call: GET /api/v1/controls.
- Add Control button must open a modal/slide-over form with fields: Control ID (auto-generated), Control Title, Control Type (Preventive / Detective / Corrective — dropdown with plain-language description), Nature (Manual / Automated / Hybrid), Design Effectiveness slider (0-100%), Operating Effectiveness slider (0-100%), Control Description, Control Owner (user select), Review Frequency (Monthly/Quarterly/Annually).
- The Edit button must open the same form pre-populated.
- Add a 'Linked Risks' column showing how many risks this control is currently linked to, with a clickable count that shows a popover list of those risk codes.

Control type plain-language descriptions

- Preventive: Stops the risk from happening in the first place. Example: Multi-Factor Authentication prevents unauthorised access.
- Detective: Identifies when a risk event has occurred. Example: SIEM alerts detect when unusual login patterns are happening.
- Corrective: Helps recover after a risk event has occurred. Example: Backup restoration recovers data after a ransomware attack.

5.6 Treatment Monitor (src/pages/TreatmentMonitor.tsx)

What must change

- Replace static treatment plans with API call: GET /api/v1/treatments.

- Add status filter tabs at the top: All | Draft | Submitted | Approved | In Progress | Overdue | Closed.
- The Overdue tab must auto-populate with any treatment plans where deadline < today and status is not Closed.
- Add an Update Progress button per row. Clicking it opens an inline form to enter a progress percentage (0-100) and an update note. Submitting this calls PUT /api/v1/treatments/:id/progress and creates an audit log entry.
- Show a deadline countdown for each in-progress treatment: e.g. '12 days remaining' in green, '2 days remaining' in red.
- Add a bulk action toolbar: select multiple treatments and assign to a new owner, or change status.

5.7 KRI Module (src/pages/KRIs.tsx)

What must change

- Replace static KRI array with API call: GET /api/v1/kris.
- The RAG status (Red/Amber/Green) must be computed dynamically: if actual_value >= red_threshold then Red; if actual_value >= amber_threshold then Amber; else Green. This logic must run on the server when actual_value is updated, and the result stored as a rag_status field.
- Add a Trend Sparkline column: a small inline line chart (7 data points — last 7 readings) showing whether the KRI is improving, stable, or deteriorating.
- When a KRI turns Red, display a red banner notification at the top of the page: 'KRI Alert: [KRI Title] has breached its critical threshold. Current value: [X]. Threshold: [Y]. Owner: [Name].'
- Create KRI button must open a form: KRI Code (auto-generated, linked to risk code), Title, Indicator description, Formula/Data Source, Red Threshold, Amber Threshold, Green Threshold, Measurement Frequency (Daily/Weekly/Monthly), Owner (user select), Linked Risk (search and select).
- Add an Update Value button per KRI row that allows manual entry of the latest actual value, with a timestamp and note. This triggers the RAG recalculation and, if breached, sends a notification to the KRI owner and the risk owner.

6. Document Management — Manual and AI-Powered Extraction

The Documents module must support two modes of file handling: manual upload with categorisation, and AI-powered extraction where the system reads uploaded documents and automatically populates risk fields. This is one of the most valuable features for consultants and auditors who deal with large volumes of policy documents, audit reports, and vendor assessments.

6.1 Manual File Upload

What to build

- The Documents page must show a drag-and-drop upload zone at the top. Supported file types: PDF, DOCX, XLSX, PNG, JPG. Maximum file size: 25MB per file.
- When a file is uploaded, the user must see a categorisation form: Document Type (dropdown: Policy, Procedure, Audit Report, Risk Assessment, Control Evidence, Vendor Report, Regulatory Guidance, Other), Linked Risk (optional — search and select a risk code), Description (textarea), Review Date (date picker).
- After categorisation, the file is uploaded to cloud storage (S3 or Azure Blob) via POST /api/v1/documents/upload. The API returns a document_id and a pre-signed URL for viewing.
- The document library must show cards for each document with: file icon, document name, type badge, linked risk code (if any), uploaded by, upload date, and action buttons (View, Download, Delete, Link to Risk).
- Add category filter tabs at the top: All | Framework | Policies | Audit Reports | Evidence | Risk Appetite Statements.

6.2 AI-Powered Document Extraction

This feature allows a user to upload a risk assessment report, an audit finding document, or a vendor questionnaire, and have the system automatically extract risk data and pre-populate a new risk form. This saves consultants significant manual data entry time.

How it works — plain language

> *Think of this as having a junior analyst read the document first and fill in the risk form for you. You review their work, correct anything that is wrong, and approve it. The AI does the reading; the consultant does the judgement.*

Extraction workflow

1. User uploads a document (PDF or DOCX) and checks the 'Extract Risk Data' checkbox in the categorisation form.
2. The backend receives the file, sends it to an AI extraction service (Claude API or GPT-4o) with a structured prompt.
3. The AI returns a JSON object with extracted fields: risk_title, risk_description, risk_category, likelihood (1-5), impact (1-5), existing_controls[], recommended_treatment, regulatory_references[].

4. The front-end opens the New Risk form pre-populated with the extracted fields. All fields are editable. A banner at the top reads: 'These fields were extracted from [filename] using AI. Please review and correct before submitting.'
5. Fields that the AI was not confident about are highlighted in amber with a warning icon and tooltip: 'AI confidence: Low — please verify this field.'
6. The user reviews, edits, and submits the form as normal. The submitted risk includes a document_source field referencing the original file.

AI Extraction Prompt Template

Implementation instruction: Send the following system prompt to the AI model when a document is uploaded for extraction. Replace [DOCUMENT_TEXT] with the extracted text from the uploaded file.

```
System: You are a GRC risk analyst. Extract structured risk data from the following
document.
Return ONLY valid JSON with these exact keys:
{ "risk_title": string,
  "risk_description": string,
  "risk_category": "Strategic"|"Operational"|"Financial"|"Compliance"|"Reputational",
  "likelihood": 1|2|3|4|5,
  "impact": 1|2|3|4|5,
  "existing_controls": string[],
  "recommended_treatment": "Avoid"|"Mitigate"|"Transfer"|"Accept",
  "regulatory_references": string[],
  "confidence_flags": { [field]: "High"|"Medium"|"Low" } }
```

7. Approvals Workflow & Audit Trail

7.1 Approval Workflow — How It Must Work

The Approvals page (currently showing as 'Admin Center') must be repurposed into a functional approval queue. Every risk submitted for approval, every treatment plan requiring sign-off, and every KRI threshold change must flow through this queue.

Approval States and Transitions

From Status	Action	To Status	Who Can Perform	Notification Sent To
Draft	Submit for Approval	Pending Approval	Risk Analyst, Risk Owner	Assigned Approver
Pending Approval	Approve	Active	Compliance Officer, CRO, Admin	Submitter, Risk Owner
Pending Approval	Reject with Comments	Draft (with rejection note)	Compliance Officer, CRO, Admin	Submitter, Risk Owner
Active	Request Review	Under Review	Any user	Risk Owner, Compliance Officer
Active	Close Risk	Closed	CRO, Admin only	All stakeholders on risk

Approvals Page UI

- Show a tabbed queue: Pending My Approval | Submitted by Me | All (admin only).
- Each item in the queue shows: risk code, risk title, risk level badge, submitted by (name + avatar), submitted date, days waiting (auto-calculated, shown in red if > 5 days).
- Clicking an item expands it to show the full risk summary inline without leaving the page.
- Approve button: opens a modal with a comment field (optional). Clicking Confirm Approval calls POST /api/v1/risks/:id/approve.
- Reject button: opens a modal with a mandatory rejection reason text field. The rejection reason is stored and displayed on the risk detail page.

7.2 Audit Trail — Non-Negotiable Requirements

> *For auditors: The audit trail is not a nice-to-have feature. It is a legal and regulatory requirement under SOX, ISO 27001, and GDPR. Every change to any risk record must be recorded immutably with four pieces of information: who made the change, what they changed, what the old value was, and what the new value was. This record must never be deletable by any user, including administrators.*

What must be logged

Action	Entity	Fields Logged	Example Log Entry

CREATE	Risk	All fields in initial state	[2026-05-25 09:14] Ahmed Al-Rashid (Risk Analyst) created risk EB-42: Vendor Data Leak
UPDATE	Risk	Changed field name, old value, new value	[2026-05-26 11:32] Sarah Chen (Compliance Officer) updated likelihood: 3 to 4 on risk EB-42
SUBMIT	Risk	Submitter, timestamp, note	[2026-05-26 14:00] Ahmed Al-Rashid submitted EB-42 for approval. Note: All fields verified.
APPROVE	Risk	Approver, timestamp, comment	[2026-05-27 09:05] Nora Al-Ansari (CRO) approved risk EB-42. Comment: Accepted with monitoring.
REJECT	Risk	Rejector, reason, timestamp	[2026-05-27 10:12] Nora Al-Ansari rejected EB-42. Reason: Likelihood score requires re-assessment with IT team.
LINK	Control to Risk	Control ID, weight, actor	[2026-05-28 08:45] Sarah Chen linked CTRL-001 (MFA) to risk EB-42 with weight 0.7
UPLOAD	Document	Filename, type, linked entity	[2026-05-28 11:00] Ahmed uploaded Vendor_Assessment_Q2.pdf as Control Evidence for risk EB-42

Audit Trail Viewer — UI Specification

- Accessible from the Activity Log tab on any Risk Detail page.
- Also accessible as a standalone page under Admin > Audit Logs (admin and auditor roles only).
- Display in reverse chronological order. Each entry on a timeline — a vertical line with a coloured dot (green = create/approve, amber = update, red = reject, blue = link/upload).
- Each entry expands on click to show the full diff: old value crossed out in red, new value shown in green.
- Export button: exports the audit trail for a specific risk or date range as a CSV or PDF report.

8. Role-Based Access Control (RBAC) Matrix

Every page and every action in the application must be gated by the user's role. The following matrix defines exactly what each role can see and do. Implement this as a permissions object in src/lib/permissions.ts that every page and API endpoint checks against.

Action	Risk Analyst	Risk Owner	Compliance Officer	Internal Auditor	CRO	Admin
View risk register	Own dept	Own risks	All	All (read)	All	All
Create new risk	Yes	Yes	Yes	No	Yes	Yes
Edit risk (active)	Own only	Own only	All	No	All	All
Submit for approval	Yes	Yes	Yes	No	Yes	Yes
Approve / Reject risk	No	No	Yes	No	Yes	Yes
Create / Edit controls	Yes	No	Yes	No	Yes	Yes
View audit trail	Own risks	Own risks	All	All	All	All
Upload documents	Yes	Yes	Yes	Evidence only	Yes	Yes
Delete documents	Own uploads	No	All	No	All	All
Generate reports	Own dept	Own risks	All	All	All	All
Manage users / admin	No	No	No	No	Limited	Full

9. Reports — Making Every Tile Functional

The Reports page currently shows 12 report type tiles, none of which generate actual output. Each must become functional. Below is the specification for each report type, what it must contain, and how it must be generated.

Report Title	What It Contains	Filters Available	Format
All Risks	Full risk register with all fields: IRS, RRS, treatment status, owner, last updated.	Department, Division, Level, Date Range	PDF, XLSX
Key Risk Report	Only risks marked as Key Risks (Critical and High level). Executive summary format with 1-page overview.	Division, Date Range	PDF only
Appetite Breached	Risks where Residual Risk Score exceeds the defined Risk Appetite threshold for that category.	Category, Division	PDF, XLSX
Comprehensive Risk Report	Single risk full detail: all fields, linked controls, treatment plan history, KRIs, audit trail, documents.	Select one Risk Code	PDF only
Enterprise Risk Summary	Board-level 1-2 page summary: total risks, distribution by level, top 5 critical risks, treatment progress, KRI breaches.	Date (point-in-time)	PDF, PPT
Departmental Summary	Risk profile for one department: all risks, scores, treatment status, KRI status.	Select Department	PDF, XLSX
KRI Breached Report	All KRIs in Red or Amber status. Shows actual value, threshold, breach date, owner, response action taken.	Date Range, Severity	PDF, XLSX
Root Cause Report	Groups risks by root cause category. Shows frequency of each cause and the risks associated with it.	Category, Date Range	PDF, XLSX

9.1 Report Generation Technical Approach

- Use a server-side PDF library (Puppeteer, WeasyPrint, or PDFKit) to render reports. Never generate reports client-side — they must be consistent regardless of the user's browser or screen size.
- Each report type corresponds to a GET /api/v1/reports/:type endpoint with a format query parameter (pdf or xlsx).
- The API endpoint runs the data query, renders an HTML template, converts it to PDF, and streams the binary file to the client with the correct Content-Type and Content-Disposition headers for download.
- Every generated report must include in its footer: report name, generation date and time, generated by (user name and role), and filter parameters used. This is required for auditability.

10. Implementation Roadmap — What to Build First

The following phased plan organises all the work described in this document into a logical build sequence. Each phase builds on the previous one. Do not begin Phase 2 work until Phase 1 is complete and tested.

Phase	Name	What to Build	Definition of Done
1	**Foundation**	7. Backend API scaffold with PostgreSQL 8. JWT authentication + RBAC 9. Replace risk scoring engine (risk-utils.ts) 10. Wire Risk Register to real API 11. Audit log table + logging middleware	Login works. Risks persist across page refresh. Scores are correct. Every write action creates an audit log entry.
2	**Core Workflows**	12. New Risk form (all steps, real submission) 13. Approval workflow + Approvals queue page 14. Risk Detail — all 4 tabs functional 15. Control Library — create, edit, link to risk 16. Treatment Monitor — state machine + progress updates	A risk can be created, submitted, approved, linked to controls, and have a treatment plan created and updated — end to end.
3	**Intelligence Layer**	17. Document upload + manual categorisation 18. AI document extraction + pre-population 19. KRI threshold engine + RAG alerts 20. Risk Heat Map on Dashboard 21. Notification system (in-app + email)	A user can upload a PDF, the system extracts risk data, the user reviews and submits it. KRI breaches generate visible alerts.
4	**Reporting & Integration**	22. All 8 report types functional (PDF + XLSX) 23. Regulatory framework tagging on all risks 24. OpenAPI spec published for GRC Wisdom 25. Multi-tenancy (tenant_id on all tables) 26. Admin Centre — user management, org setup	Any report can be generated and downloaded. The module can be handed to a different organisation (multi-tenant). API is documented and ready for GRC Wisdom.

10.1 Key Risks in the Build (Meta-Level)

These are the risks in building this module itself — shared transparently so the development team can plan mitigations.

Build Risk	Impact if Not Mitigated	Mitigation
Scoring engine inconsistency between frontend and backend	Dashboard shows different scores to the risk register. Auditors cannot trust the numbers.	Run scoring server-side only. Frontend calls the API to get scores; it never computes them independently.
Audit log gaps if middleware is incomplete	Regulatory non-compliance. Auditors find unlogged changes.	Log at the service layer, not the API layer. Every database write must go through a service that logs before writing.
AI extraction hallucinations	Wrong risk scores or categories pre-populated without the user noticing.	Always require user review before submission. Highlight low-confidence fields. Never auto-submit extracted data.
File storage security	Evidence files (audit reports, vendor assessments) exposed to unauthorised users.	Use pre-signed URLs with 15-minute expiry for all document access. Never expose direct S3/Blob paths.

This document covers every gap, every feature, and every workflow the VS Code agent needs to make this module production-ready. Build Phase 1 first. Test it thoroughly. Then proceed to Phase 2.